



Inżynieria Oprogramowania

dr Katarzyna Grzesiak-Kopeć





07

Refaktoryzacja



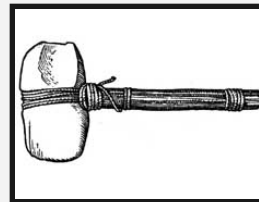
Zmiany to norma!



NOWE

**Wymagania
Technologia
Błędy
Optymalizacja**

**Rozwiązania dobre dzisiaj nie muszą
być dobre jutro!**



Zmiany w zespole utrzymania



Oprogramowanie często działa przy domyślnych założeniach



Założenia są w głowach deweloperów

- > wiedza o procesie
- > wiedza o produkcie
- > strata ludzi to strata wiedzy



Zmiany technologii



Nowe sposoby przechowywania danych
pliki
relacyjne bazy danych
bazy danych NoSQL



Paradygmaty programowania
> strukturalne
> obiektowe
> funkcyjne



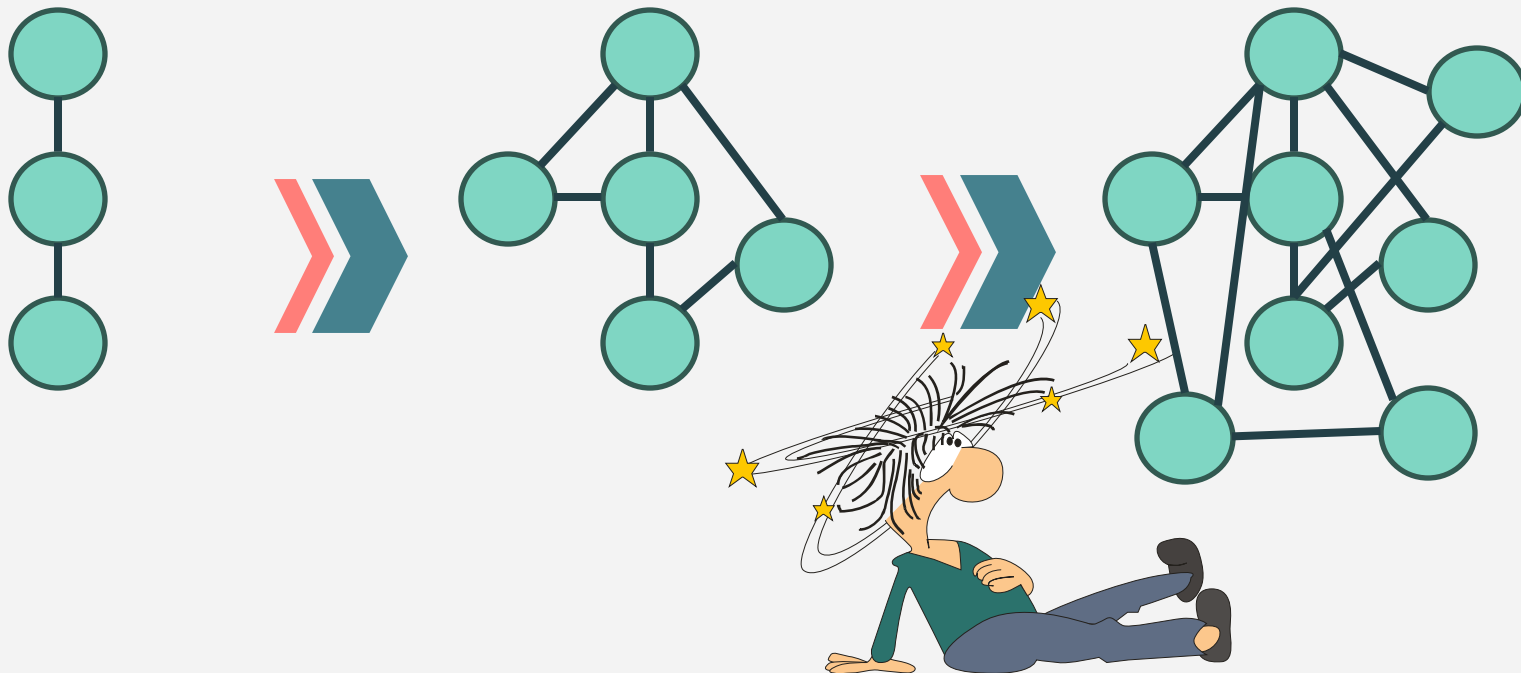
Infrastruktura
chmura obliczeniowa



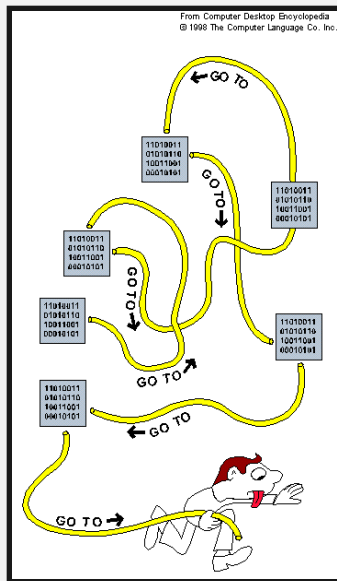
Nowe języki programowania i urządzenia peryferyjne



Zmiany wewnętrzne



Spaghetti code



- ✓ Określenie pejoratywne
- ✓ Kod trudny do zrozumienia, utrzymania i poprawy
- ✓ Kod zawiera wiele instrukcji typu
 - ✓ GOTO,
 - ✓ wyjątki lub inne „nieustrukturyzowane” instrukcje rozgałęziające wątek kontroli

<https://jtech.digital/pasta-programming>

THE PASTAS OF PROGRAMMING

Pasta Theory -- describing different programming structures as popular pasta dishes while highlighting the shortcomings of the code.



Spaghetti

WHAT
Complex, tangled structure. The use of many unstructured branching constructs, such as GOTO statements, exceptions, threads, or other.

CAUSE
Unstructured code-writing or modifications by too many people over a long period of time.

HOW TO AVOID
Structured programming greatly decreases the incidence of spaghetti code.

When faced with Spaghetti code, it is in your best interest to take the time to refactor, creating a cleaner implementation.



Ravioli

WHAT
Small and loosely coupled software components

CAUSE
Creating small code modules that work to accomplish a greater objective in a program's source code.

GOODS
Some argue it's a good strategy if documented correctly.

BADS
Ravioli code is only an issue if you overdo it, creating modules that are too small.

Many feel that ravioli code is neither innately good or bad, and should be considered case-by-case.



Lasagna

WHAT
Well-defined, separable layers of code. Each layer accesses services in the layers below through well-defined interfaces.

USES
Client-server applications are frequently lasagna code with well-defined interfaces between client and server.

GOODS
While the overall system is more complex, the individual layers are easier to manage.

There is no problem in computer science that cannot be solved by adding another layer of indirection, except having too many layers of indirection.
-David Wheeler



Macaroni

WHAT
Uses a mixture of computer languages in a single document.

USES
Various programming languages have the functionality of working in mixed environment in a single document. For example, in web programming, it is common to layer HTML, JavaScript, SQL and PHP. This usually occurs out of necessity in order to provide a user with the full experience.

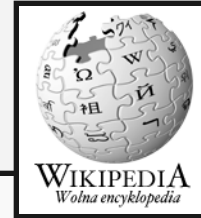
BADS
Difficult to see the big picture which may exist only in a single programmer's head.

Infographic by JTech Communications



Spaghetti code

```
10 i = 0
20 i = i + 1
30 if i <= 10 then goto 80
40 if i > 10 then goto 60
50 goto 20
60 print "Program Completed."
70 end
80 print i; " squared = "; i * i
90 goto 20
```



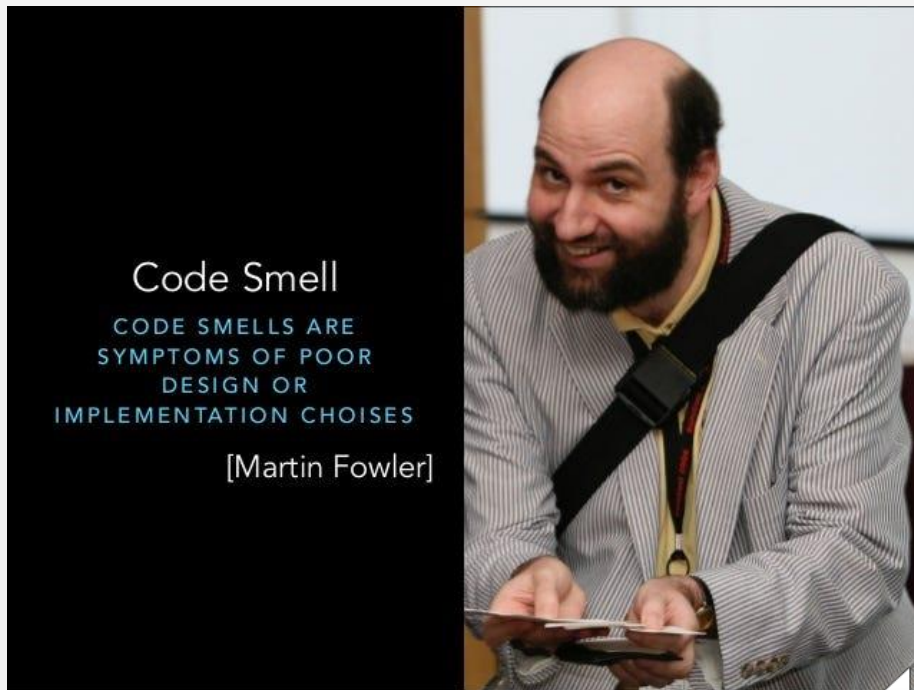
Jak sobie radzić?

CODE SMELLS

Ucz się
Znajdź
Popraw



REFAKTORYZACJA



<https://medium.com/oceanize-geeks/code-smells-and-refactoring-c2c0e064>

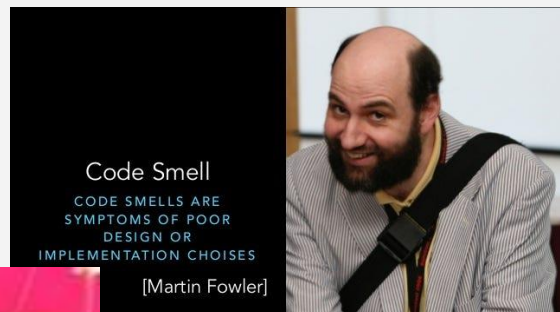
Jak sobie radzić?

CODE SMELLS

Ucz się
Znajdź
Popraw



REFAKTORYZACJA



William F. (Bill) Opdyke



Kent Beck

Refaktoryzacja



To zmiana organizacji **struktury** kodu programu nie zmieniająca jego działania.



Refaktoryzacja

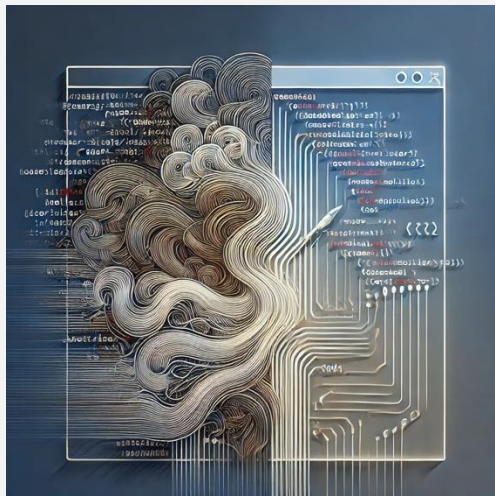


To zmiana organizacji **struktury** kodu programu nie zmieniająca jego działania.

$$x^2 - x - 2$$



Refaktoryzacja



To zmiana organizacji **struktury** kodu programu nie zmieniająca jego działania.

$$x^2 - x - 2$$



$$(x+1)(x-2)$$



Refaktoryzacja



Martin Fowler

A studied, described, and catalogued coding technique for changing a small part of the design of code **without changing the external behavior** of the code.

A strategy for code maintenance by means of applying refactorings freely whenever a need is discovered.
A separate activity from adding new functionality.



Po co refaktoryzacja?



Ulepsza projekt oprogramowania
Często prowizorycznie zmieniamy kod



Łatwiej zrozumieć oprogramowanie
Czytelniejszy kod



Pomaga lokalizować błędy
Czyni strukturę czytelniejszą odkrywając błędy



Pomaga szybciej programować
Co wynika z poprzednich punktów



• Jakie języki można refaktoryzować? •



Jakie języki można refaktoryzować?

WSZYSTKIE

Obiektowe można
refaktoryzować na
więcej sposobów



Kiedy refaktoryzować?

- ⚙️ Przed dodaniem funkcjonalności
- ⚙️ Kiedy trzeba poprawić błędy
- ⚙️ W trakcie inspekcji kodu
- ⚙️ Jeśli robisz to samo po raz trzeci
- ⚙️ Kiedy trafiasz na **bad smell**



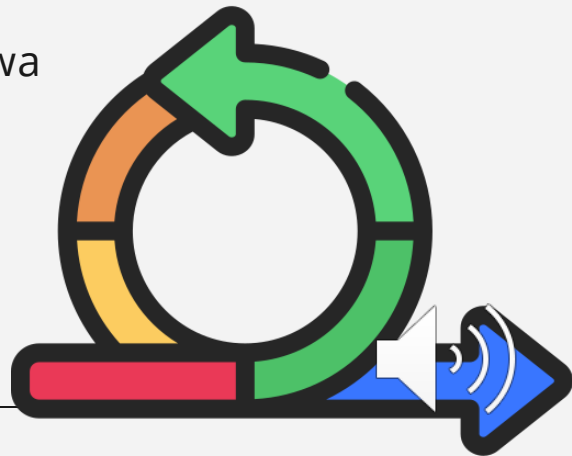
Bad smells

- ⚙️ Powtarzający się kod
Stwórz metodę
- ⚙️ Długa lista parametrów
Wprowadź obiekt jako parametr
- ⚙️ Instrukcje **switch**
Zastosuj polimorfizm
- ⚙️ I mnóstwo innych:
<https://refactoring.com/catalog/>



Jak refaktoryzować?

- 01** Zaprojektuj dużą liczbę testów dla kodu, który ma być poddany analizie
- 02** Zidentyfikuj problemy: **bad smells**
- 03** Zrefaktoryzuj
- 04** Przetestuj
- 05** Jeśli nie przejdzie wszystkich testów to zacznij od nowa



Jak refaktoryzować?

01 Małe kroki

projektuj mało, koduj mało, zmieniaj mało i zawsze testuj

02 Odtwarzalne

by inni mogli je zrozumieć

03 Ogólne

reguły, które można stosować

04 Spisane

wiemy co krok po kroku i gdzie było zastosowane

05 Zaczynamy od elementów najbardziej ryzykownych



Przykład

EXTRACT METHOD

01 Nazwa

Extract Method

02 Opis sytuacji

Fragment kodu, który może zostać wydzielony. Uczyń z tego kodu metodę i odpowiednio ją nazwij

03 Motywacja

zastosuj Extract Method kiedy istniejące metody są zbyt długie lub istnieją w kodzie powtórzenia

04 Mechanizm

05 Przykład



Przykład

EXTRACT METHOD

04 Mechanizm

- > Utwórz nową metodę i odpowiednio ją nazwij
- > Skopiuj kod do tworzonej metody
- > Ustal zmienne lokalne
- > Jeśli zmieniasz jedną zmienną lokalną, to uczyn ją wartością zwracaną, jeśli więcej, to może konieczna jest dalsza refaktoryzacja
- > Zmienne lokalne, których nie zmieniasz, uczyn parametrami wejściowymi
- > Skompiluj
- > Zastąp w kodzie źródłowym skopiowany kod odpowiednim wywołaniem nowej metody
- > Skompiluj i przetestuj



Przykład

EXTRACT METHOD

01 Nazwa

Extract Method

02 Opis sytuacji

Fragment kodu, który może zostać wydzielony. Uczyń z tego kodu metodę i odpowiednią nazwij

03 Motywacja

zastosuj Extract Method kiedy istniejące metody są zbyt długie lub istnieją w kodzie powtórzenia

04 Mechanizm

05 Przykład



Przykład

EXTRACT METHOD

05 Przykład



```
void printOwing() {  
    printBanner();  
  
    //print details  
    System.out.println ("name:      " + _name);  
    System.out.println ("amount    " + getOutstanding())  
}
```

```
void printOwing() {  
    printBanner();  
    printDetails(getOutstanding());  
}  
  
void printDetails (double outstanding) {  
    System.out.println ("name:      " + _name);  
    System.out.println ("amount    " + outstanding);  
}
```



Przykład

RENAME METHOD

- ✓ Nazwa metody nic nie mówi o jej działaniu
- ✓ Zmień nazwę

```
//find the ID of the organization employing this participant  
String oid = orgsInDB.getOrganization(participantId);
```

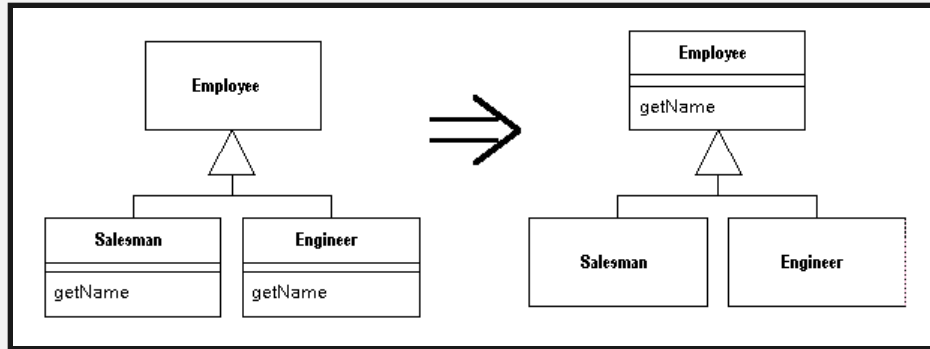
```
String oid = orgsInDB.findOrganizationEmploying(participantId);
```



Przykład

PULL UP METHOD

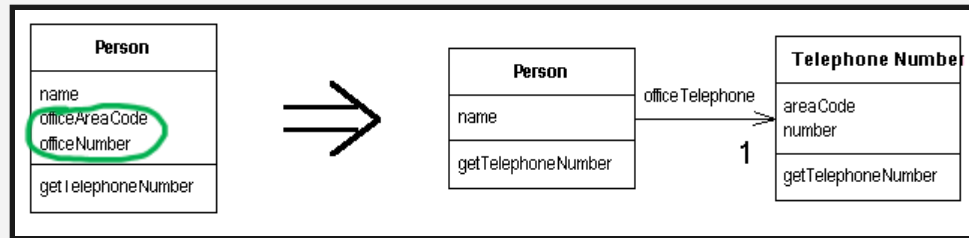
- ✓ Są metody robiące to samo w podklasach
- ✓ Przesuń je w górę hierarchii



Przykład

EXTRACT CLASS

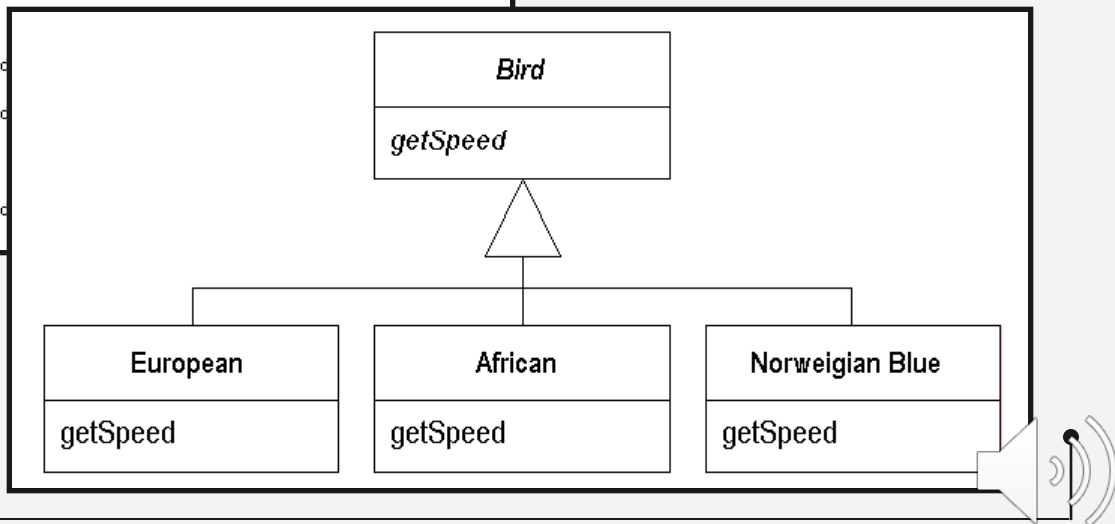
- ✓ Jedna klasa „robi” to, co powinny dwie
- ✓ Stwórz nową klasę i przenieś do niej odpowiednie atrybuty oraz metody



Przykład REPLACE CONDITIONAL WITH POLYMORPHISM

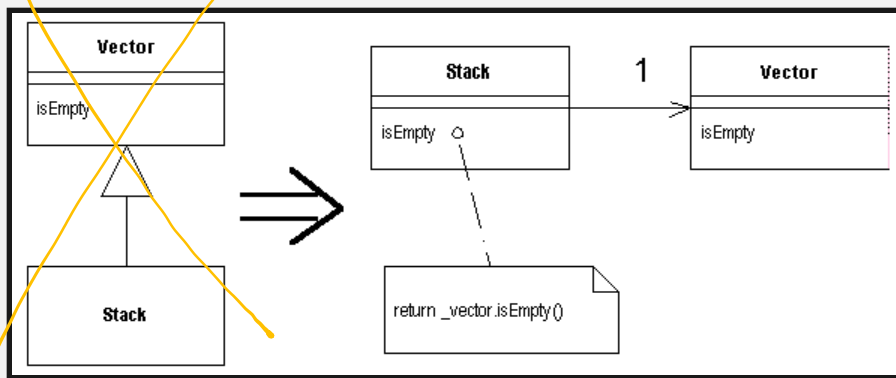
- ✓ Tzw. **TYPE-CODE**
- ✓ Przenieś każdy z warunków do przeciążonej metody w podklasie
- ✓ Z oryginalnej metody zrób **abstrakcyjną**

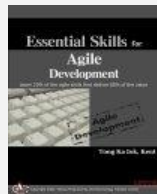
```
double getSpeed() {  
    switch (_type) {  
        case EUROPEAN:  
            return getBaseSpeed  
        case AFRICAN:  
            return getBaseSpeed  
        case NORWEIGIAN_BLUE:  
            return (_isNailed)  
    }  
    throw new RuntimeException ("Should")  
}
```



Przykład REPLACE INHERITANCE WITH DELEGATION

- ✓ Podklasa wykorzystuje tylko część interfejsu klasy nadrzędnej albo nie chce dziedziczyć danych
- ✓ Utwórz pole dla klasy nadrzędnej, dostosuj metody delegowane i usuń dziedziczenie





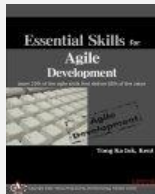
```
class Shape {  
    final static int TYPELINE = 0;  
    final static int TYPEPERECTANGLE = 1;  
    final static int TYPECIRCLE = 2;  
    final static int TYPETRIANGLE = 3;  
    int shapeType;  
    Point p1;  
    Point p2;  
    //third point of the triangle.  
    Point p3;  
    int radius;  
}
```

TYPE-CODE

```
class CADApp {  
    void drawShapes(Graphics graphics, Shape shapes[]) {  
        for (int i = 0; i < shapes.length; i++) {  
            switch (shapes[i].getType()) {  
                case Shape.TYPELINE:  
                    graphics.drawLine(shapes[i].getP1(), shapes[i].getP2());  
                    break;  
                case Shape.TYPEPERECTANGLE:  
                    //draw the four edges.  
                    graphics.drawLine(...);  
                    graphics.drawLine(...);  
                    graphics.drawLine(...);  
                    graphics.drawLine(...);  
                    break;  
                case Shape.TYPECIRCLE:  
                    graphics.drawCircle(shapes[i].getP1(), shapes[i].getRadius());  
                    break;  
                case Shape.TYPETRIANGLE:  
                    graphics.drawLine(shapes[i].getP1(), shapes[i].getP2());  
                    graphics.drawLine(shapes[i].getP2(), shapes[i].getP3());  
                    graphics.drawLine(shapes[i].getP3(), shapes[i].getP1());  
                    break;  
            }  
        }  
    }  
}
```



Jak usunąć TYPE-CODE?

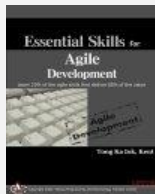


```
class Shape {  
}  
class Line extends Shape {  
    Point startPoint;  
    Point endPoint;  
}  
class Rectangle extends Shape {  
    Point lowerLeftCorner;  
    Point upperRightCorner;  
}  
class Circle {  
    Point center;  
    int radius;  
}
```

```
class CADApp {  
    void drawShapes(Graphics graphics, Shape shapes[]) {  
        for (int i = 0; i < shapes.length; i++) {  
            if (shapes[i] instanceof Line) {  
                Line line = (Line) shapes[i];  
                graphics.drawLine(line.getStartPoint(), line.getEndPoint());  
            } else if (shapes[i] instanceof Rectangle) {  
                Rectangle rect = (Rectangle) shapes[i];  
                graphics.drawLine(...);  
                graphics.drawLine(...);  
                graphics.drawLine(...);  
                graphics.drawLine(...);  
            } else if (shapes[i] instanceof Circle) {  
                Circle circle = (Circle) shapes[i];  
            }  
        }  
    }  
}
```



Jak usunąć TYPE-CODE?



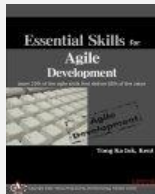
```
class Shape {  
}  
class Line extends Shape {  
    Point startPoint;  
    Point endPoint;  
}  
class Rectangle extends Shape {  
    Point lowerLeftCorner;  
    Point upperRightCorner;  
}  
class Circle extends Shape {  
    Point center;  
    int radius;  
}
```

```
class CADApp {  
    void drawShapes(Graphics g)  
    for (int i = 0; i < shapes.length; i++)  
        shapes[i].draw(g);  
}
```

```
abstract class Shape {  
    abstract void draw(Graphics graphics);  
}  
class Line extends Shape {  
    Point startPoint;  
    Point endPoint;  
    void draw(Graphics graphics) {  
        graphics.drawLine(getStartPoint(), getEndPoint());  
    }  
}  
class Rectangle extends Shape {  
    Point lowerLeftCorner;  
    Point upperRightCorner;  
    void draw(Graphics graphics) {  
        graphics.drawLine(...);  
        graphics.drawLine(...);  
        graphics.drawLine(...);  
        graphics.drawLine(...);  
    }  
}  
class Circle extends Shape {  
    Point center;  
    int radius;  
    void draw(Graphics graphics) {  
        graphics.drawCircle(getCenter(), getRadius());  
    }  
}
```



Jak usunąć TYPE-CODE?

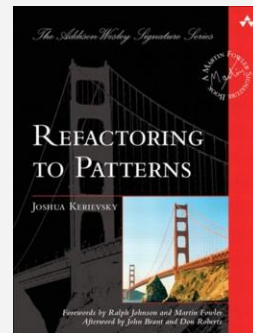


```
interface Shape {  
    void draw(Graphics graphics);  
}  
class Line implements Shape {  
    ...  
}  
class Rectangle implements Shape {  
    ...  
}  
class Circle implements Shape {  
    ...  
}
```

```
class CADApp {  
    void drawShapes(Graphics graphics, Shape shapes[]) {  
        for (int i = 0; i < shapes.length; i++) {  
            shapes[i].draw(graphics);  
        }  
    }  
}
```



Mariaż refaktoryzacja i wzorce



Stosowanie wzorców

By ulepszyć projekt jest lepsze niż stosowanie ich w początkowej fazie projektowania



Refaktoryzacja

Projekt może być ulepszony przez refaktoryzację wprowadzającą wzorzec projektowy



Trudności



Czas!



Czasem lepsza jest reinżynieria (przebudowa systemu)



Trudno wybrać właściwy sposób rozwiązania



Jak przewidzieć wpływ zmian lokalnych na całą architekturę?

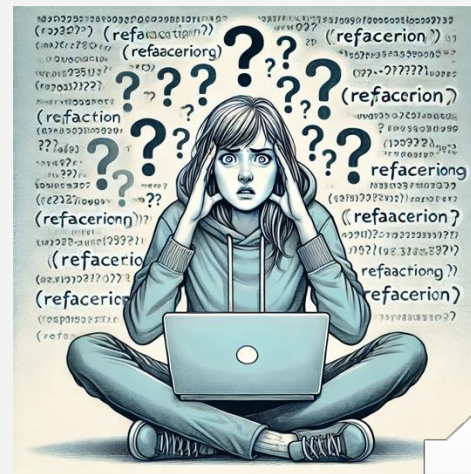


Stąd

Należy szkolić deweloperów

Doświadczenie pomaga

Kluczowe jest wsparcie narzędziowe



Koszty



Język/środowisko

Zależy od narzędzi
Generalnie znośny



Testowanie

Konieczne automatyczne testy



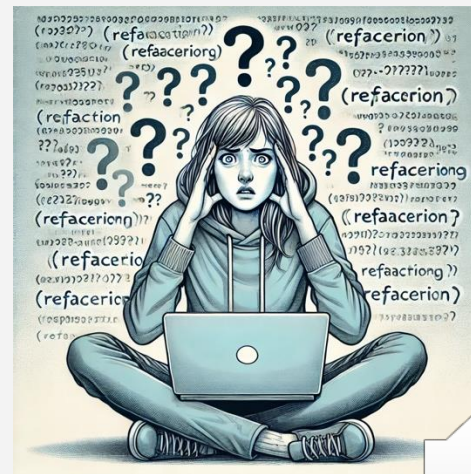
Dokumentacja

Może być dużo zmian



System

Zmiana interfejsów i odpowiedzialności to
zmiana testów





Koniec!

